

Recap Consultation

2025



Question 1. **Classes and Objects**

A function defined inside a class is called a _____ .

Correct answer: **method**

consider your smartphone.

attributes.

what are its characteristics or properties?

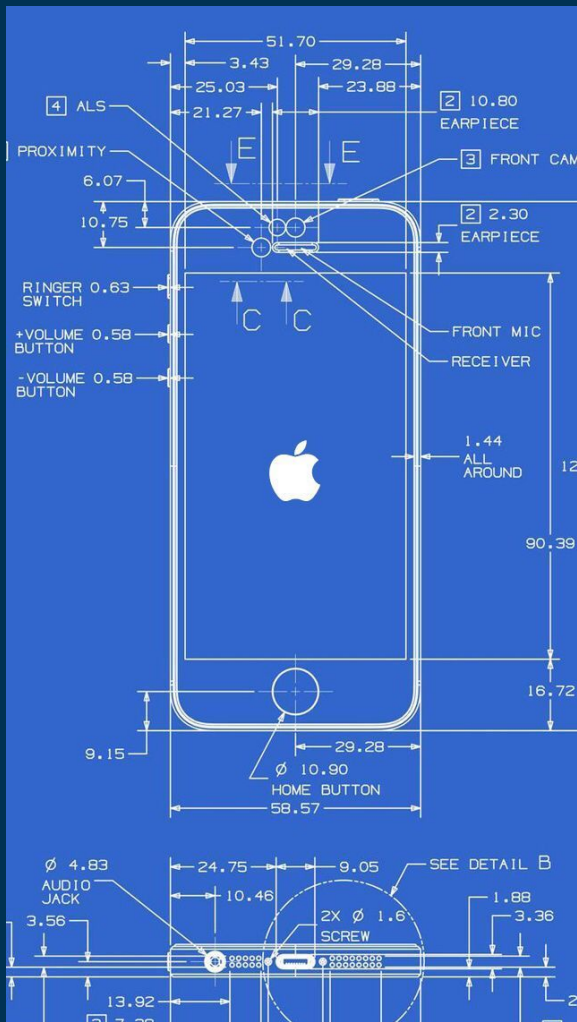
- color
- size
- brand
- battery %
- etc.



what **actions** or **operations**
can you perform with it?

- make a call
- send a text
- browse the web
- play music
- etc.

think in terms of attributes and actions.



classes:

the *blueprint* of the coding universe

think of a class in Python like a blueprint or a template for making a smartphone.

it describes:

- what a smartphone is and
- what it can do.

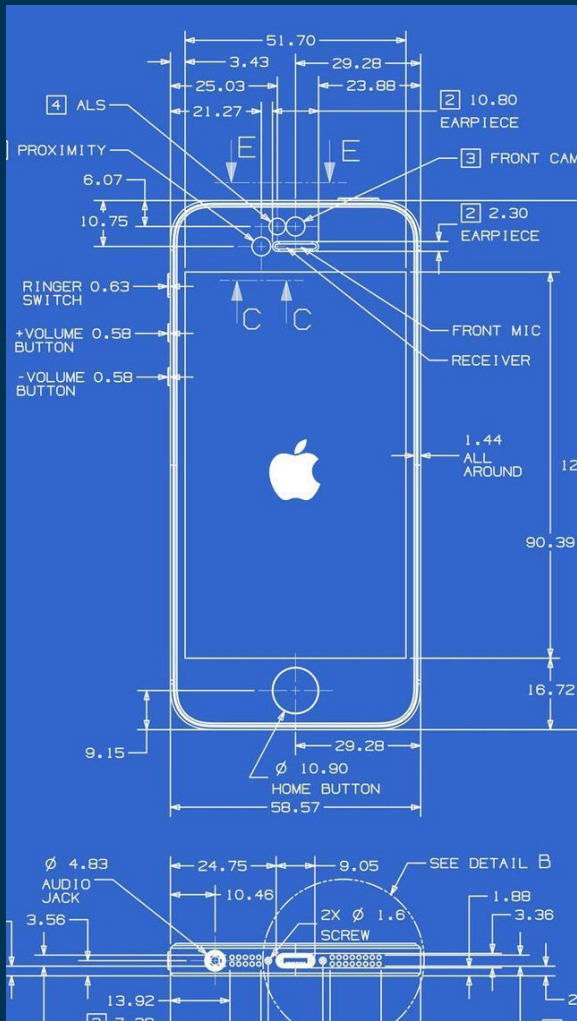
classes:

The *blueprint* of the coding universe

a **class** defines:

- **attributes** (like color, size, screen size) and
- **methods** (eg. making a call, taking photos).

however, it doesn't represent a tangible item until we create an **Object** from it.



creating classes in Python

declare a new **class** named 'Smartphone'.

attributes represent properties of the class



```
class Smartphone:

    # Attributes
    brand = "Unknown"
    color = "Black"
    battery_percentage = 100

    # Constructor Method
    def __init__(self, brand, color, battery_percentage):
        self.brand = brand
        self.color = color
        self.battery_percentage = battery_percentage

    # Action (Method)
    def make_call(self, number):
        if self.battery_percentage > 0:
            return f"Calling {number}..."
        else:
            return "Battery too low to make a call."

    # Additional Method to check battery level
    def check_battery(self):
        return f"Battery level is {self.battery_percentage}%."
```

setting default values
for attributes

constructor
initializes new
objects.

methods
represent actions.

creating objects

when our code blueprints come to life

object created from the
Smartphone class.

```
# Using the Smartphone class  
▶ my_smartphone = Smartphone("Apple", "Silver", 85)
```

providing values for attributes
during instantiation.

creating an object from a class is known as **instantiation**:

```
# instantiation syntax
```

```
object_name = ClassName(parameters)
```

instantiation is like manufacturing a product
from its blueprint

interacting with **objects**

Once an object is created, we can call its methods using the dot notation.

```
# method call notation
```

```
object_name.method_name(parameters)
```

This is like telling the object to perform a specific task.

When we call a method, the code inside that method is executed.

Question 2. Classes and Objects

Please fill a missing word:

```
class UserProfile:
    def __init__(____, username, email):
        ____ .username = username
        ____ .email = email

    def save(____):
        with open(f"{self.username}_profile.txt", "w") as file:
            file.write(f"Username: {self.username}\n Email: {self.email}")
```

Correct answer: **self**

understanding the `self` parameter

```
def __init__(self, par1, par2, ...)
```

In Python, when we talk about `self`, we're talking about the individual instance of a class.

You can think of `self` as a way for an object to **refer to itself**.

`self` is the first parameter in any method defined within a class.

It must be included in the method definition so the method can access the instance's attributes and other methods.

understanding the ``self`` parameter

Imagine each object created from a class is a person.

Using `self` is like a person talking about themselves.

Just as a person knows their own thoughts and feelings, `self` knows the object's data and can access its methods and attributes.



Question 3. Encapsulation and Access Control

What does **encapsulation** refer to in object-oriented programming?

A. The process of inheriting properties from a base class	B. The practice of keeping data and the methods that modify that data together within a class	C. Dividing a program into multiple distinct classes	D. The interaction between different objects in a program
---	---	--	---

Question 4. Encapsulation and Access Control

Which attributes will be accessed in the child class?

```
class Parent:
    def __init__(self):
        self.publicVar = "Public"
        self._protectedVar = "Protected"
        self.__privateVar = "Private"

class Child(Parent):
    def accessModifiers(self):
        print(self.publicVar)
        print(self._protectedVar)
        print(self.__privateVar)
```

A. publicVar

B. _protectedVar

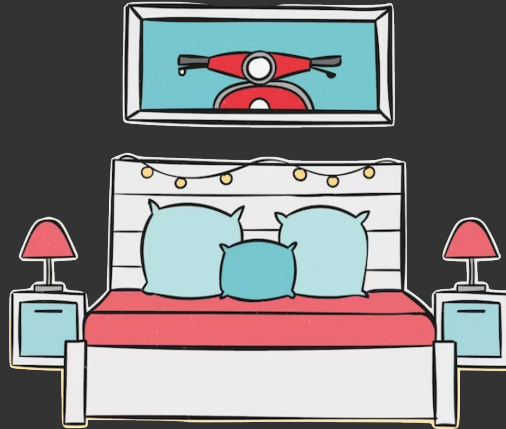
C. __privateVar

Access Control

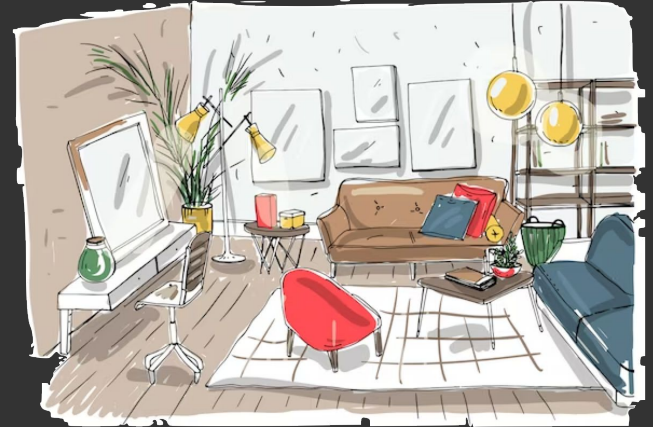
Public (Front Yard): Fully accessible to anyone.



Private (Bedrooms): Restricted access, only available to the defining class.



Protected (Living Room): Intermediate level of access, available to the defining class and its subclasses, but not to the outside world.



Access Control in Python

Public Attributes and Methods: By default, all attributes and methods in a Python class are public, meaning they can be accessed from outside the class.

Protected Attributes and Methods: By convention, a single underscore prefix (`_`) is used to denote protected attributes and methods. This indicates they are intended for internal use within the class and its subclasses, but this is purely conventional and not enforced by Python.

Private Attributes and Methods: Python uses a double underscore prefix (`__`) to denote private attributes and methods. This triggers name mangling, where Python internally changes the name of the variable in a way that makes it harder for subclasses to accidentally override private attributes and methods.

Question 5. Inheritance and Polymorphism

What are other names for **parent** class?

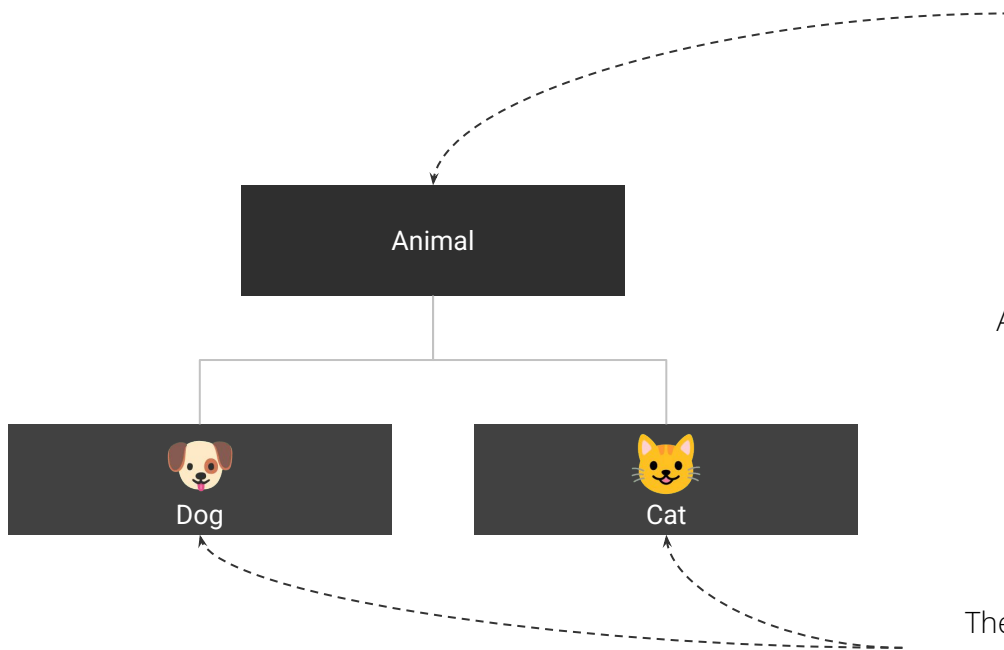
A. Base class

B. Subclass

C. Derived class

D. Superclass

inheritance



Base Class (Superclass)

The class from which properties and methods are inherited. Also known as a "**parent**" class.

Inheritance

Allows for the creation of a new class that is a **modified version** of an existing class.

Derived Class (Subclass)

The class that inherits from another class. It can add new properties and methods or modify existing ones. Also known as a "**child**" class.

inheritance

Base Class (Superclass)

The class from which properties and methods are inherited. Also known as a "**parent**" class.

```
class Animal: # Parent class
    def eat(self):
        print("This animal is eating")
```

```
class Dog(Animal): # Child class 🐶
    def bark(self):
        print("The dog barks")
```

```
class Cat(Animal): # Child class 🐱
    def meow(self):
        print("The cat meows")
```

Derived Class (Subclass)

The class that inherits from another class. It can add new properties and methods or modify existing ones. Also known as a "**child**" class.

understanding method **overriding**

Definition: a feature in OOP that allows a subclass to provide a **specific implementation** for a method that is already defined in its superclass.

Why Use Method Overriding:

- Customization: It allows subclasses to tailor or **modify inherited behavior** without altering the superclass code.
- Extensibility: Subclasses can **extend** the superclass's behavior by adding additional steps to the overridden method and calling the superclass's method with `super()`.
- Polymorphism: Method overriding is a key to achieving **polymorphism**, where a single interface can represent different underlying forms (data types).

How `super()` function works

In a subclass, you can call `super()` to access the superclass's methods that have been overridden in the subclass. This is particularly useful for the `__init__` method, where you want to ensure that the superclass is correctly initialized, but it applies to any method.

```
class Animal: # Parent class
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def __init__(self, name, breed):
        # Call the superclass constructor
        super().__init__(name)
        self.breed = breed
```

Question 6. Aggregation and composition

What type of relationship is represented by the code below?

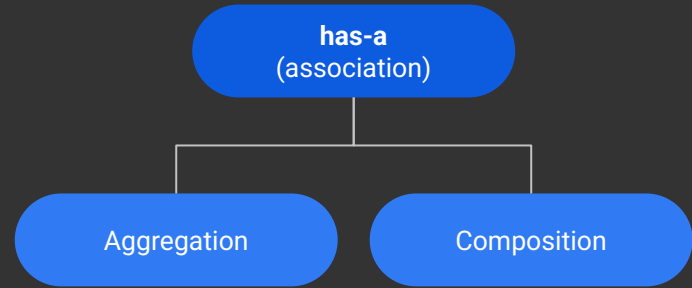
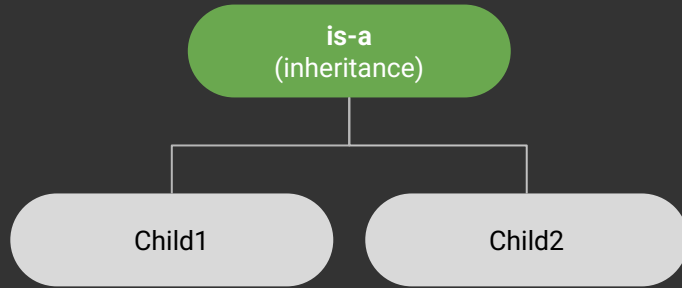
```
class Engine:
    def __init__(self, capacity):
        self.capacity = capacity

class Car:
    def __init__(self, model):
        self.model = model
        self.engine = Engine(2000)

toyota = Car("Toyota")
```

Correct answer: **composition**

types of object **relationships**



composition: example

House and Rooms:

A house is composed of rooms.

This is a classic example of composition where the rooms (the parts) **do not exist independently** of the house (the whole).

If the house were to be destroyed, the rooms would no longer exist as part of the structure.



aggregation: **example**

Relationship between a football 🏈 team and its players:

Independence: Players can move between teams through transfers, loans, or when their contracts expire. Their existence and identity as football players are not tied to a single team.

Shared Ownership: A player might play for a club team and also represent their national team in international competitions. This shows that players can be part of multiple aggregates (teams) simultaneously.

Whole-Part Relationship Without Ownership: The football team is made up of players (parts), but it doesn't own them in the sense that their careers or existence as players depend on being part of that specific team.




```

class Player:
    def __init__(self, name, position):
        self.name = name
        self.position = position

    def __str__(self):
        return f"{self.name}, Position: {self.position}"

class Team:
    def __init__(self, name):
        self.name = name
        self.players = []

    def add_player(self, player):
        self.players.append(player)

    def list_players(self):
        print(f"Team: {self.name}")
        for player in self.players:
            print(player)

    def remove_player(self, player_to_remove):
        self.players = [player for player in self.players
                        if player.name != player_to_remove.name]

class FootballTeam(Team):
    pass

class NationalTeam(Team):
    pass

```

Creating player instances

```

player1 = Player("Leo Messi", "Forward")
player2 = Player("Cristiano Ronaldo", "Forward")
player3 = Player("Luis Figo", "Forward")

```

Creating club teams and adding players

```

fc_barcelona = FootballTeam("FC Barcelona")
fc_barcelona.add_player(player1)
fc_barcelona.add_player(player3)

```

```

real_madrid = FootballTeam("Real Madrid")
real_madrid.add_player(player2)

```

Creating national teams and adding the same players

```

argentina = NationalTeam("Argentina")
argentina.add_player(player1)

```

```

portugal = NationalTeam("Portugal")
portugal.add_player(player2)
portugal.add_player(player3)

```

Listing players in club and national teams

```

real_madrid.list_players()
fc_barcelona.list_players()

```

Transfer Luis Figo to Real Madrid

```

fc_barcelona.remove_player(player3);
real_madrid.add_player(player3);

```

Question 7. Design patterns

Which of the following design patterns are classified as structural patterns?

A. Adapter

B. Builder

C. Singleton

D. Decorator

creational patterns

Focus on object creation mechanisms, trying to create objects in a manner suitable to the situation. Examples include:

- Singleton,
- Factory Method,
- Abstract Factory,
- Builder,
- and Prototype

structural patterns

Deal with object composition or the structure of classes.

These patterns help ensure that if one part of a system changes, the entire system doesn't need to do the same. Examples include:

- Adapter,
- Decorator,
- Proxy,
- Composite,
- and Bridge.

behavioral patterns

Concern how objects interact and distribute responsibility.

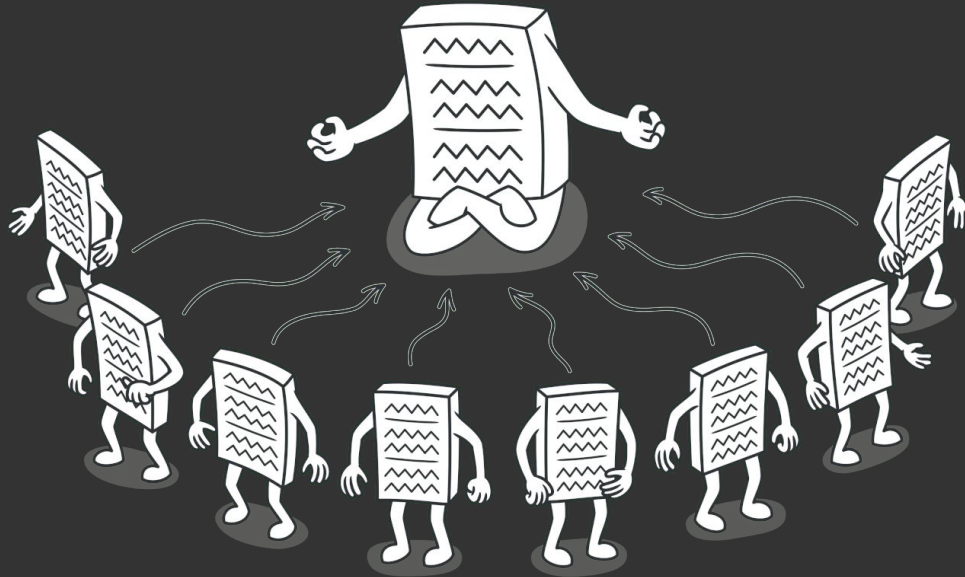
Examples include:

- Observer,
- Strategy,
- Command,
- Iterator,
- and State.

Singleton

creational patterns

Singleton is a creational design pattern that lets you ensure that a class has only one instance, while providing a global access point to this instance.



Decorator

Structural patterns

Decorator is a structural design pattern that lets you attach new behaviors to objects by placing these objects inside special wrapper objects that contain the behaviors.



Question 8. **SOLID**

Which SOLID principle states that objects of a superclass should be replaceable with objects of its subclasses without breaking the application?

A. Single Responsibility Principle	B. Open/Closed Principle	C. Liskov Substitution Principle	D. Dependency Inversion Principle
------------------------------------	--------------------------	----------------------------------	-----------------------------------

SOLID

Single
Responsibility



Liskov
Substitution



Dependency
Inversion



Open-Closed
Principle



Interface
Segregation



Single Responsibility Principle

*"There should never be more than one reason
for a class to change."*

or

*"Each software module should have one
and only one reason to change."*

or

"A module should be responsible to one, and only one, actor."

Open-Closed Principle

Main idea:

Software must be designed to allow the behavior of it to be **changed by adding new code**, rather than changing existing code.

Liskov Substitution Principle

"Subclasses should be substitutable for their base classes."

or

"Methods that use references to base classes must be able to use objects of derived classes without knowing it."

Interface Segregation Principle

"Many client specific interfaces are better than one general purpose interface."

Dependency Inversion Principle

*"Depend upon Abstractions.
Do not depend upon concretions."*

Question 9. Exception handling

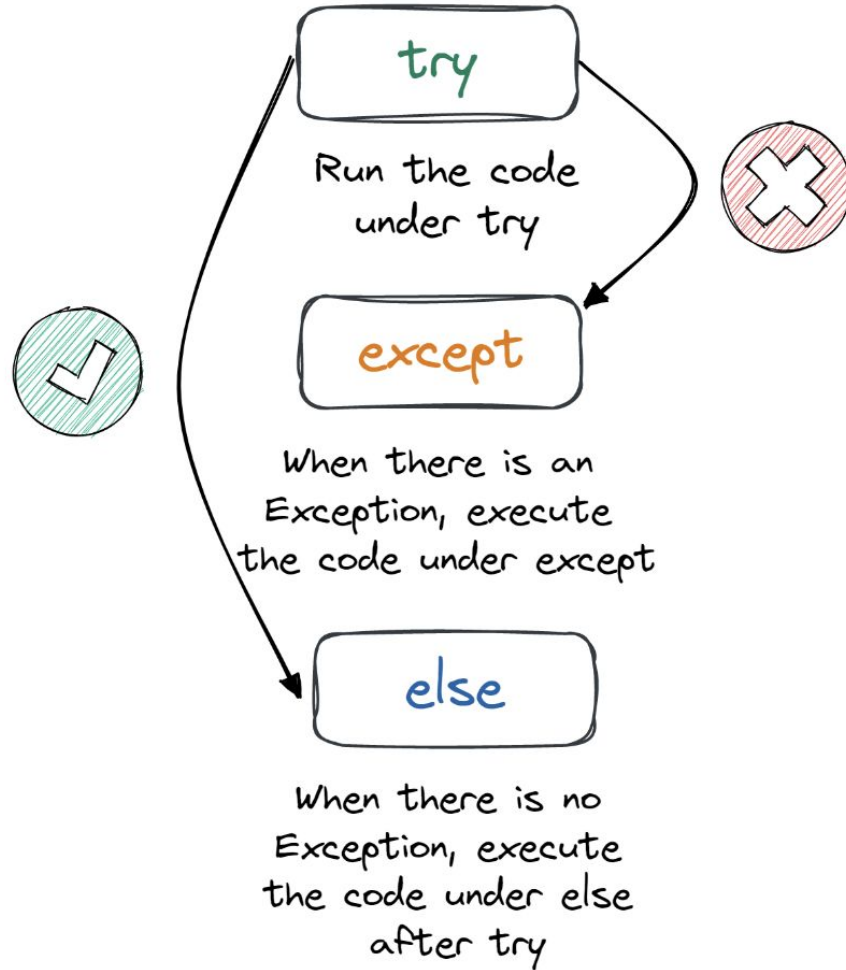
What is the output of the following code?

```
try:
    print(1 / 1)
except ZeroDivisionError:
    print("Division by zero!")
finally:
    print("This will always execute.")
```

Correct answer:

1

This will always execute.



try

except

else

finally

Always run the code
under finally.

The `finally` keyword in the try-except block is **always executed**, irrespective of whether there is an exception or not.

In simple words, the `finally` block of code is run after the try, except, the else block is final.

It is quite useful in **cleaning up** resources and closing the object, especially closing the files.

Raising exceptions

```
try:
    if age < 0:
        raise ValueError("Age cannot be negative.")
    print(f"Age set to {age}")

except ValueError as e:
    print(e)
```

This example raises a `ValueError` if a negative age is passed, which is then caught and handled in the `try-except` block.

Creating **custom** exceptions

You can also define a constructor (`__init__`) method to accept additional arguments that can describe the error in more detail:

```
class ValidationError(Exception):  
    def __init__(self, message, value):  
        self.message = message  
        self.value = value  
        super().__init__(message)
```

Question 10. Unit testing

Which of the following statements about **unittest** is true?

A. All test methods in a <code>unittest.TestCase</code> class must start with the word "test".	B. The <code>unittest</code> module is part of the standard library.	C. You must manually call each test method within the <code>unittest.TestCase</code> class.	D. The <code>unittest</code> module does not support test discovery.
--	--	---	--

Testing a Simple Function

```
# file: math_functions.py
def add_numbers(x, y):
    return x + y
```

```
# file: test_math_functions.py
import unittest
from math_functions import add_numbers

class TestMathFunctions(unittest.TestCase):
    def test_add_positive_numbers(self):
        self.assertEqual(add_numbers(3, 4), 7)

    def test_add_zero_and_negative_number(self):
        self.assertEqual(add_numbers(-1, 1), 0)

    def test_add_negative_numbers(self):
        self.assertEqual(add_numbers(-2, -2), -4)
```

Exams

Grupė	Dalykas	Kodas	Tipas	Egzamino		
				data	laikas	vieta
EAf-24	Objektinis programavimas (su kursiniu darbu)	FMSAB23201	E	2025-05-26	10:20	P2 306
EEf-24	Objektinis programavimas (su kursiniu darbu)	FMSAB23201	E	2025-05-26	08:30	P2 306
Elf-24	Objektinis programavimas (su kursiniu darbu)	FMSAB23201	E	2025-05-26	12:10	P2 306
Ef-24/1	Objektinis programavimas (su kursiniu darbu)	FMSAB23201	E	2025-05-27	08:30	P2 306
Ef-24/2	Objektinis programavimas (su kursiniu darbu)	FMSAB23201	E	2025-05-27	10:20	P2 306
EKf-24	Objektinis programavimas (su kursiniu darbu)	FMSAB23201	E	2025-05-27	12:10	P2 306
EDIf-24/1	Objektinis programavimas (su kursiniu darbu)	FMSAB23201	E	2025-05-30	12:10	P2 203
EDIf-24/2	Objektinis programavimas (su kursiniu darbu)	FMSAB23201	E	2025-05-30	10:20	P2 203

thank you! it was a great pleasure!

<https://www.linkedin.com/in/tomasz-szturo-7b43a686/>

tomasz.szturo@visma.com

A man in a green jacket is riding a bicycle away from the camera down a dirt path. The path is lined with large, leafy trees on the right and a hedge on the left. The scene is bright and sunny, with shadows cast on the path.

Entrepreneurial Responsible Dedicated Inclusive

Champions of cloud software

